

AI STUDY BUDDY

AI-Powered Personalized Learning Assistant

TEAM ID	SWTID-2026-1506
Project Domain	Artificial Intelligence / Education Technology
Document Type	Project Documentation

Prepared using the supplied sample project documentation as a structural reference.

Abstract

AI Study Buddy is a proposed AI-powered learning assistant designed to help students understand academic topics, revise lessons, and practice concepts through a simple conversational interface. Users can enter a topic or question, request an easy explanation, generate concise notes, and create practice questions. The system uses a generative AI model through an API, while a web application manages input validation, prompt construction, and presentation of responses. The goal is to make self-study more interactive, accessible, and time-efficient.

1. Introduction

Students often spend considerable time searching for explanations, preparing revision notes, and creating practice questions. AI Study Buddy brings these learning activities together in one application. It acts as a supplementary study tool and does not replace teachers, textbooks, or verified course material.

1.1 Project Objectives

- Provide clear, student-friendly explanations of user-selected topics.
- Generate concise revision notes and key-point summaries.
- Create quizzes and practice questions for self-assessment.
- Allow users to select an explanation style or difficulty level.
- Offer a responsive interface for desktop and mobile devices.

1.2 Scope

The initial version focuses on text-based learning assistance. It accepts a subject, topic, question, and optional learning preferences, then returns explanations, notes, or quiz content. Account management, saved learning history, file uploads, and multilingual speech support may be considered for later versions.

2. Project Description

AI Study Buddy uses generative AI to provide personalized study support. Its main modules are: Topic Explainer, which presents a concept in simple language; Notes Generator, which organizes a topic into concise headings and bullet points; Quiz Generator, which creates practice questions with answers; and Study Guidance, which suggests a short revision plan based on the topic and available time.

2.1 Example Scenarios

- A student enters "Photosynthesis" and selects Beginner. The system explains the process step by step and highlights important terms.
- A student preparing for an examination enters a chapter name and requests revision notes. The system returns an organized summary for quick review.
- A student selects Quiz mode for a topic. The system generates multiple-choice questions and an answer key for self-assessment.

3. Technical Architecture

The application follows a three-tier architecture. The frontend collects user requests and displays results. The backend validates input, prepares prompts, communicates with the AI service, and formats the response. The

AI service generates learning content based on the supplied topic and instructions.

Layer	Responsibilities
Frontend	HTML, CSS and JavaScript interface; topic entry; mode and difficulty selection; result display.
Backend	Python with Flask; routing; input validation; prompt construction; error handling; API integration.
AI Service	Generates explanations, notes, quizzes and study suggestions from structured prompts.
Deployment	Local Flask server for development; optional public tunnel or hosted deployment for demonstration.

3.1 Data Flow

- User enters a topic/question and selects a feature and difficulty.
- Frontend sends the request to the backend using an HTTP POST request.
- Backend checks required fields, length limits, and supported options.
- Backend builds a task-specific prompt and sends it to the configured AI API.
- The response is checked and formatted before being returned to the frontend.
- The interface displays the result and provides options to copy or retry.

The initial design does not require a database. If user accounts, saved notes, or learning history are introduced, database tables and an ER diagram should be added.

4. Prerequisites

- Python 3.8 or later and pip.
- Basic knowledge of Python, HTML, CSS and JavaScript.
- Flask framework and a virtual environment (venv).
- Access to an AI provider API and an API key, if using a hosted model.
- Git for version control.
- A modern web browser and internet connection for API-based generation.

5. Project Workflow

Milestone 1: Requirements and Model Selection

- Identify student needs and define Explain, Notes, Quiz and Study Plan modes.
- Review available AI models for response quality, latency, context size and cost.
- Choose a model suitable for educational explanations and structured outputs.
- Create the project environment and store API credentials securely.

Milestone 2: Core Feature Development

- Design prompt templates for each learning mode.
- Define input fields such as topic, question, difficulty and preferred response length.
- Implement validation and clear error messages.
- Specify a consistent response structure for each mode.

Milestone 3: Backend Development

- Create the Flask application and home route.
- Implement a generation endpoint that accepts JSON input.
- Construct prompts from the selected mode and learning preferences.
- Call the AI service, handle failures, and return structured JSON.
- Keep API keys in environment variables and exclude secret files from version control.

Milestone 4: Frontend Development

- Build a clean, responsive interface with a topic input area and mode selector.
- Show loading feedback while a response is being generated.
- Display explanations, notes and quizzes in readable sections.
- Add copy and regenerate controls where appropriate.
- Ensure the interface works on mobile, tablet and desktop screens.

Milestone 5: Testing and Deployment

- Test empty input, long input, unsupported modes and API errors.
- Check that generated content matches the selected mode and difficulty.
- Run the application locally and verify the complete user flow.
- Deploy using a suitable hosting service or temporary secure tunnel for demonstrations.
- Do not expose API keys or sensitive student information in logs or public repositories.

Milestone 6: Evaluation and Enhancement

- Collect feedback on clarity, usefulness and ease of use.
- Improve prompts when answers are vague, too advanced or incorrectly formatted.
- Plan future enhancements such as saved study history, file-based questions and multilingual support.

6. Functional Requirements

- The system shall accept a topic or question from the user.
- The system shall allow the user to choose a learning mode.
- The system shall provide optional difficulty or explanation-style selection.
- The system shall validate input before calling the AI service.
- The system shall display generated content in a readable format.

- The system shall show an understandable error message when generation fails.

7. Non-Functional Requirements

- Usability: simple navigation and clear labels.
- Performance: return results within a reasonable time, subject to network and model response times.
- Reliability: handle invalid input and API failures without crashing.
- Security: protect API keys and avoid collecting unnecessary personal data.
- Compatibility: support current desktop and mobile browsers.
- Maintainability: keep frontend, backend and prompt logic organized into separate components.

8. Testing

Test Case	Expected Result
Submit blank topic	Validation message; AI request is not sent.
Request an explanation	A clear, topic-related explanation is displayed.
Request revision notes	Notes appear with headings and key points.
Generate a quiz	Questions and answer key are returned in the requested format.
Simulate API failure	A friendly error message appears and the interface remains usable.
Use on a mobile screen	Controls and results fit the screen and remain readable.

9. Responsible Use and Limitations

AI-generated content can contain mistakes, omissions, or outdated information. Students should verify important facts against class notes, textbooks, and teacher guidance. Generated material is a study aid rather than an authoritative source. The system should not request unnecessary personal data, and API credentials must remain private.

10. Future Enhancements

- User login and secure saved study history.
- Upload notes or PDF documents to generate summaries and quizzes from course material.
- Multilingual explanations and translation.
- Adaptive quizzes based on previous answers.
- Progress tracking and personalized revision schedules.
- Text-to-speech and accessibility improvements.

11. Conclusion

AI Study Buddy brings explanations, revision notes, quizzes and study guidance into a single AI-assisted learning experience. By combining a responsive frontend, a structured backend and a generative AI service,

the project aims to make independent study more organized and engaging. With careful validation, responsible use, and future improvements, it can develop into a useful companion for students across different subjects and learning levels.